09 — Writing GPU Kernels: CUDA and cuBLAS

How a GPU runs your code, why the order in which you touch memory matters more than the arithmetic, and how a hand-written matrix multiplication compares with NVIDIA's library.

Reading time: about 50 minutes. You need: modules 00 to 05, especially section 2 of module 05, and the ability to read C or C++. You do not need: a GPU to read this article or to do half of the exercise.

Two kernels, identical arithmetic

Picture two GPU kernels that multiply two matrices. They perform exactly the same additions and multiplications, in the same order. The only difference is which of each thread's two index coordinates selects a row and which selects a column.

That one swap changes how the GPU reads memory. In one kernel, a group of 32 threads reads 32 neighbouring numbers, which the hardware fetches in a single transaction. In the other, the same 32 threads read numbers that are far apart, and the hardware needs 32 separate transactions. The second kernel moves 32 times as many bytes to do identical work. Published measurements, such as Simon Boehm's worklog, show this kind of change making a kernel several times faster.

That is the lesson of this whole module. Near the hardware, speed is mostly about moving data, not about arithmetic. It is also the hidden reason behind several earlier results: why decoding is limited by memory (module 05), and why FlashAttention wins (module 07).

What you will learn

By the end of this article, you will be able to:

- describe how a CUDA kernel's threads are organised, and why groups of 32 matter;
- explain the GPU memory hierarchy, and the single rule behind every optimisation in this module;
- climb the matrix-multiplication ladder, from a naive kernel to a register-tiled one;
- write reductions, such as a sum or a softmax, efficiently;
- call cuBLAS correctly despite its column-major layout, and benchmark without fooling yourself.

Section 1 explains why kernels matter. Sections 2 and 3 cover the execution model and memory. Sections 4 to 7 climb from matrix multiplication to fused attention. A recap and the exercise briefing follow at the end.

---

1. Why a research engineer writes kernels

Most people at an AI lab never write a GPU kernel. Those who do have unusual leverage: a 20% improvement in the attention kernel is a 20% saving on every training run and every generated token.

For interviews, the bigger benefit is understanding. Several claims from earlier modules are really statements about the GPU's memory. Decoding is memory-bound, the KV cache is the binding constraint, and FlashAttention wins by moving less data. You can take them on trust, or you can write the kernels and see why they hold.

This module also completes the course's running thread. Attention has appeared as tensor code, as mathematics, as hand-written gradients, as a bug, as a distributed computation, as a latency problem and as a

paper. Here it becomes a GPU kernel.

---

2. How a kernel runs

Grids, blocks and threads

A **kernel** is a function that runs on the GPU, many copies at once. Launching a kernel creates a **grid** of **thread blocks**. Each block runs on one **streaming multiprocessor** (SM) and contains up to 1,024 **threads**. Threads in the same block can share memory and wait for each other, using `__syncthreads()`. Threads in different blocks essentially cannot.

Each thread computes its own position from built-in variables, then works on its own slice of data. Here is the core of the vector-addition kernel from the exercise:

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
if (i < n) c[i] = a[i] + b[i];
```

The first line gives each thread a unique global index. The check `i < n` matters because the data length is rarely an exact multiple of the block size. Writing past the end corrupts memory silently.

Warps: the most important number

Threads are scheduled in groups of 32 called **warps**. The 32 threads of a warp run in lockstep: one instruction, applied to 32 lanes at once. Two consequences follow, and between them they explain most of what makes GPU code fast or slow.

Divergence. If threads in the same warp take different branches of an `if`, the warp executes both branches, with the unused lanes masked off. A condition such as `if (threadIdx.x % 2)` can halve throughput. Branches that differ only *between* warps cost nothing.

Coalescing. When the 32 lanes of a warp read memory, the hardware combines their reads into as few 128-byte transactions as possible. Thirty-two numbers of 4 bytes each fill exactly one 128-byte line, if they are neighbours. The exercise counts transactions for several access patterns:

Access pattern within one warp	Transactions	Useful bytes ÷ bytes fetched
32 neighbouring numbers	1	100%
every 32nd number	32	3.1%
naive matrix multiply, reading along a row of A	32	3.1%
same kernel with indices swapped, reading B	1	100%

A kernel with poor coalescing requests the same bytes but fetches 32 times as many. This is the most common performance bug in hand-written kernels, and the fix is usually swapping two indices.

---

3. The memory hierarchy

A GPU has several kinds of memory, from tiny and very fast to large and slow. The figures below are rounded, and cover recent NVIDIA data-centre parts; the exact numbers for one architecture are in NVIDIA's tuning guide for it.

Level	Size	Access time	Bandwidth
registers	about 256 KB per SM	about 1 cycle	very high
shared memory	100–228 KB per SM	about 30 cycles	about 10 TB/s
L2	40–50 MB	about 200 cycles	about 5 TB/s
main memory (HBM, high-bandwidth memory)	40–192 GB	400–800 cycles	2–8 TB/s

Every optimisation in this module follows one rule: **move data up the hierarchy once, then use it many times.** Tiling a matrix follows this rule. So does FlashAttention, and so does the KV cache.

Bank conflicts

Shared memory has a detail worth knowing. It is divided into 32 banks of 4 bytes each. If two threads in the same warp access different addresses in the same bank, their accesses happen one after the other instead of together.

A tile declared as `float tile[32][32]` and read column by column hits the same bank every time: a 32-way conflict. The standard fix is to pad it to `float tile[32][33]`. That shifts each row by one bank, at a cost of 128 bytes.

Occupancy, and why maximising it is not the goal

Occupancy is the number of warps a streaming multiprocessor can hold at once, compared with its maximum. It is limited by whichever runs out first: threads, registers per thread, or shared memory per block. The exercise includes a calculator. A modest kernel with 256 threads per block reaches full occupancy. Give each thread four times as many registers, and occupancy falls to 25%, with registers as the limit.

High occupancy usually helps, because it gives the scheduler more work to interleave. But it is not the goal. The register-tiled matrix multiply in section 4 deliberately uses many registers per thread, which lowers occupancy, and it is faster anyway. The work done within each thread more than compensates. Volkov's talk "Better Performance at Lower Occupancy" is the classic reference.

---

4. The matrix-multiplication ladder

Matrix multiplication, called **SGEMM** in its library form, is the standard way to learn all this. The name is BLAS shorthand: S for single precision, GE for a general matrix, MM for multiply. Each optimisation applies one idea, and NVIDIA's cuBLAS library provides a ceiling to measure against.

Recall **arithmetic intensity** from module 05: floating-point operations per byte moved. A naive kernel for $n \times n$ matrices performs about $2n^3$ operations while reading about $2n^3$ numbers of 4 bytes each, an intensity of only 0.25. Perfect reuse would read each matrix once instead, which is $3n^2$ numbers, giving $2n^3/12n^2 = n/6$. At $n = 4{,}096$ that is 683, above an H100's ridge point of 296. So the naive kernel is slow because of memory traffic, not because the problem is hard.

Level 0: naive. Each thread computes one element of the result, reading a whole row of A and a whole column of B from main memory. The row index comes from `threadIdx.x`, so the 32 lanes of a warp read 32 different rows of A: the worst coalescing pattern from section 2.

Level 1: coalesced. Swap the indices so that `threadIdx.x` selects the column instead. Now the 32 lanes read 32 neighbouring entries of B in one transaction. The arithmetic is unchanged. In Boehm's measurements, this step alone makes the kernel several times faster.

Level 2: shared-memory tiling. Load a 32×32 tile of A and of B into shared memory, call `__syncthreads()`, and compute that part of the product from the fast copies. Each element now comes from main memory 32 times less often, so the arithmetic intensity rises from 0.25 to 8.

Level 3: register tiling. Each thread now computes a small patch of the result, 4×4 in the exercise's kernel, holding its running sums in registers. Each value copied into shared memory is then used several times instead of once. It is the same tiling idea, applied one level further up the hierarchy.

Level 4: the last steps. Loading 16 bytes per instruction instead of 4, fetching the next tile while computing the current one, and using the GPU's tensor cores can close most of the remaining gap. Beyond this point you are competing with NVIDIA's CUTLASS library, and the sensible engineering choice is to use it.

In Boehm's published worklog on one GPU, a naive kernel reached roughly 1% of cuBLAS's speed, and a kernel with two-dimensional register tiling reached roughly two-thirds. Running `02_sgemm.cu` on an RTX 3060 at $n = 1024$ reproduces that shape:

Level	GFLOP/s	Share of cuBLAS
cuBLAS	6869	100%
0 naive, uncoalesced	100	1.5%
1 coalesced	787	11.5%
2 shared-memory tiled	979	14.3%
3 register tiled 4×4	4850	70.6%

Levels 0 and 1 do identical arithmetic, and coalescing alone is worth 7.8 times. The step that pays most is level 3, which is the one that stops going to memory for every multiply. Your own numbers will depend on your GPU, and the exercise measures them.

The exercise also verifies each tiling scheme's index arithmetic without a GPU. Emulations in numpy reproduce the exact product for tiled and register-tiled kernels, including matrices whose sizes are not multiples of the tile size. They also confirm that every element of the result is written exactly once.

> **In the interview:** the shape of the ladder is the lesson. The gains come from the memory hierarchy, in order from the bottom up, and the first gain, coalescing, costs nothing.

---

5. Reductions and warp shuffles

Many operations reduce many numbers to one: a sum, a maximum, the statistics of a LayerNorm. They share a standard structure in four levels:

1. **Within a thread:** loop over several elements, adding into a register.
2. **Within a warp:** combine the 32 lanes with `__shfl_down_sync`, which passes register values directly between lanes, with no memory traffic. Five rounds reduce 32 values to 1.
3. **Within a block:** write one value per warp to shared memory, then have one warp reduce those.
4. **Across the grid:** use a second kernel launch, or an atomic operation.

The last level hides a subtle point. Adding floating-point numbers in a different order can give a slightly different result, and atomic operations run in an unpredictable order. So an atomic sum is not deterministic. This is one reason `torch.use_deterministic_algorithms(True)` makes some operations slower.

Softmax, again

A numerically stable softmax needs a maximum, then a sum, then a normalising pass. Written naively, that reads each row three times. The online softmax from module 07 needs fewer passes over the data, because it carries the running maximum and sum together.

Fewer passes should mean less time, since softmax is memory-bound, but the measurement is more interesting than that. On an RTX 3060 over a 4096×4096 input, the three-pass kernel took 0.78 ms and the two-pass kernel took 0.78 ms as well; only the online kernel moved, to 0.61 ms. A row is 16 KB, so a re-read lands in cache rather than in memory, and the rule "fewer passes means less time" holds against DRAM rather than against reads. The exercise's `03_softmax.cu` compares the versions, and asks what a truly single-pass version would need.

---

6. cuBLAS, and the column-major trap

cuBLAS is NVIDIA's library of linear-algebra kernels. You will not beat it at matrix multiplication. The point of racing it is to learn how far behind you are, and why.

Column-major storage

The trap that catches almost everyone is memory layout. cuBLAS stores matrices **column by column**, a convention inherited from Fortran. C and PyTorch store them **row by row**. The same bytes describe different matrices under the two conventions.

A small example makes this concrete. Take a 2×3 matrix A and a 3×2 matrix B, stored row by row:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \qquad B = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \qquad C = AB = \begin{bmatrix} 4 & 5 \\ 10 & 11 \end{bmatrix}$$

Hand A's six bytes to cuBLAS as a 3×2 matrix. Reading column by column, cuBLAS sees $\left[\begin{smallmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{smallmatrix}\right]$, which is exactly $A^\top$. In the same way, it sees B as $B^\top$.

That suggests the standard trick. Ask cuBLAS for $B^\top A^\top$, by passing B first and A second with the dimensions swapped. The answer, $B^\top A^\top = (AB)^\top = C^\top$, is stored column by column. Read back row by row, its bytes give exactly C. A numpy check confirms every step.

In the exercise's kernel file, the call looks like this:

```
cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N,
            N, M, K, &alpha,
            d_B, N,        /* B passed first */
            d_A, K,        /* then A */
            &beta, d_C, N);
```

Get this wrong, and cuBLAS returns either a transposed answer or a matrix of plausible-looking wrong numbers, with no error.

A few more essentials:

- Create one cuBLAS handle and reuse it; creating handles is slow.
- For mixed precision and tensor cores, use `cublasGemmEx` or cuBLASLt.
- `cublasSetMathMode` can enable TF32, which uses fewer significant bits than float32. Results then differ slightly from float32, which is expected rather than a bug.
- Batched versions, such as `cublasSgemmStridedBatched`, exist for the batched matrix multiplications in attention.

Benchmarking without fooling yourself

Three rules catch almost everyone at least once:

1. **Warm up first.** The first kernel launch pays for setup and compilation.
2. **Synchronise before reading the clock.** Kernel launches return immediately, before the work is done. Without `cudaDeviceSynchronize()` or CUDA timing events, you measure how fast work was queued, not how fast it ran.
3. **Report a fraction of the best, not only a time.** "My kernel takes 4 ms" says little. "My kernel reaches 38% of cuBLAS" is an engineering statement.

Then profile rather than guess. `nsys` shows a timeline of everything the program does, and `ncu` reports counters for each kernel. The first numbers to check are occupancy and memory throughput.

---

7. Fusing attention

The last kernel brings the module together. The exercise's `04_attention.cu` computes attention the FlashAttention way. It keeps tiles of the queries, keys and values in shared memory, and computes scores in registers. It applies the online softmax as each tile of keys arrives, and writes the output once. The full score matrix never reaches main memory.

The notebook verifies this kernel's logic with an emulation. It matched standard attention to within 7.2×10^{-16} without a mask, and 2.0×10^{-15} with a causal mask. The largest block of scores held at once stayed at 32 numbers, while the full matrix grew from 1,024 to 9,216 numbers.

This kernel will be slower than PyTorch's fused attention, and its source file lists why, in order of cost:

1. It uses one warp per block, which wastes most of each streaming multiprocessor's capacity.
2. It does not use tensor cores.
3. It does not fetch the next tile while computing the current one.
4. It loads one number per instruction instead of several.
5. It processes one query row per block, instead of reusing each key-and-value tile across many rows.

That list is the point. Compare two answers in an interview. "I reached a third of the fused kernel's speed, and the remaining gap is tensor cores and prefetching" is honest and specific. It is far more convincing than a vague claim of expertise.

Recap

- A kernel runs as many threads, scheduled in warps of 32 that execute in lockstep.
- Coalescing combines a warp's memory reads. Reading neighbours needs one transaction; reading far apart needs 32.
- Every optimisation follows one rule: move data up the memory hierarchy once, then use it many times.
- The matrix-multiplication ladder climbs through coalescing, shared-memory tiling and register tiling, in that order.
- cuBLAS stores matrices column by column. Pass B before A, with dimensions swapped, to get a row-major product.
- Benchmark with warm-up and synchronisation, and report speed as a fraction of the best available.

The exercise: two halves

What you will build. The exercise has two halves, and both are worth doing.

- **The GPU-free half, in the notebook.** Calculators for arithmetic intensity and occupancy, a counter for coalescing transactions, and numpy emulations of the exact index arithmetic in the CUDA kernels. Kernel bugs are almost always indexing bugs, and indexing can be checked on any computer.
- **The CUDA half, in the `kernels/` folder.** Four programs: vector addition, the matrix-multiplication ladder raced against cuBLAS, softmax reductions, and fused attention. Each checks itself against a CPU reference and prints a timing table.

A note on the CUDA half: all four programs have now been built and run, on an RTX 3060 with CUDA 13.2 under Windows. They compile clean and pass their own checks. Two defects surfaced on that first real run, and both are worth knowing about, because they are the kind this module is about:

- `cudaDeviceProp::memoryClockRate` was removed in CUDA 13, so `common.cuh` would not compile. It now reads the figure through `cudaDeviceGetAttribute`, which works from CUDA 11 onwards.
- `04_attention.cu` built its test inputs with `size_t` arithmetic, so a subtraction below zero wrapped to about 1.8×10^{19}. The attention kernel and its CPU reference agreed on that input and the check passed, but the softmax had saturated to one-hot, so the test proved much less than it appeared to. With the inputs corrected, the kernel still passes, and now it means something.

The second one is the better story. A check that passes is not the same as a check that tests anything, and the way to tell them apart is to look at the numbers it printed.

If a program fails to build, or reports `FAIL`, the notebook's emulations verify the same index arithmetic without a GPU, which helps you tell an indexing mistake from a problem elsewhere.

Time and setup. Allow three hours for the GPU-free half, and three to four hours for the CUDA half. The CUDA half needs the CUDA toolkit and a C++ compiler. On Linux, run `make` in `kernels/`. On Windows, run `.\build.ps1 -Run`, which needs the Microsoft C++ build tools, because the CUDA compiler relies on them.

How to run it. Open `09-cuda-and-cublas/exercise.ipynb`, or run `python exercise.py` from inside the module folder. Part 6 of the notebook runs any kernels you have built.

How the checks work. They work as in module 00. Before you start, the notebook lists any built kernels, then each part reports that it is not implemented:

```
built binaries: []
[FAIL] part 1  -- NotImplementedError:
```

```
0/1 passed.  Still failing: part 1
```

Step map. This table links each part to the section of the article it practises.

Part	You write or run	The idea it practises	Section
1	`ridge_point`, `attainable_flops`, `naive_gemm_intensity`, `tiled_gemm_intensity`	arithmetic intensity	4
2	`transactions_per_warp`, `coalescing_efficiency`	coalescing	2
3	`occupancy`	occupancy and its limits	3
4	`emulate_tiled_gemm`, `emulate_register_tiled_gemm`, `gemm_write_map`	the tiling ladder	4
5	`emulate_flash_kernel`	fused attention	7
6	build and run the four programs in `kernels/`	the whole module, on real hardware	2 to 7

When you are done. Every GPU-free check passes. Your emulations reproduce `A @ B` exactly, including for matrices whose sizes are not multiples of the tile, and write every output element exactly once. If you built the kernels, record your matrix-multiplication results in the table the notebook provides, as a fraction of cuBLAS for each level of the ladder.

If you get stuck.

- For coalescing, write out the addresses the 32 lanes of one warp read, then group them into 128-byte lines by hand.
- For the tiling emulations, start with a matrix exactly one tile in size, where the tiled result must equal the ordinary product.
- If a kernel fails to build on Windows with "cannot find compiler cl.exe", install the Visual Studio C++ build tools. `build.ps1` then finds them automatically.
- If you are still stuck, `solution.py` has a complete reference for the GPU-free half.

Want more? The notebook lists six exercises inside the kernels themselves. Examples include adding 16-byte loads to the matrix multiply and making the softmax truly single-pass. Others process several query rows per block in the attention kernel, or rewrite that kernel in Triton for comparison.

Going deeper

Start here

- Boehm (2022), *How to Optimize a CUDA Matmul Kernel for cuBLAS-like Performance: a Worklog* — https://siboehm.com/articles/22/CUDA-MMM. Every step of the ladder in section 4, with measured speeds. The best single companion to this module.

NVIDIA documentation

- NVIDIA, *CUDA C++ Programming Guide* — https://docs.nvidia.com/cuda/cuda-c-programming-guide/. Chapter 2 covers the execution model, and chapter 5 covers performance.
- NVIDIA, *CUDA C++ Best Practices Guide* — https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/. Section 9, on memory, is the reference for coalescing.

- NVIDIA, *cuBLAS Library User Guide* — https://docs.nvidia.com/cuda/cublas/. Read section 2.1, on column-major storage, before writing your first call.
- NVIDIA, *CUTLASS* — https://github.com/NVIDIA/cutlass. What a production-quality matrix multiply looks like.

Kernel design

- Volkov (2010), *Better Performance at Lower Occupancy*, GTC. Why register pressure can be worth more than occupancy.
- Harris, *Optimizing Parallel Reduction in CUDA*, NVIDIA. The classic step-by-step optimisation of a reduction kernel.
- Luitjens (2014), *Faster Parallel Reductions on Kepler*, NVIDIA developer blog. The warp-shuffle version of reductions.
- Kerr, Diamos & Yalamanchili (2010), *Dynamic Compilation of Data-Parallel Kernels*. Background on why naive kernels perform so poorly.

Attention on the GPU

- Milakov & Gimelshein (2018), *Online normalizer calculation for softmax*. arXiv:1805.02867.
- Dao et al. (2022), *FlashAttention*. arXiv:2205.14135. Reread section 3.1 after this module; with the memory hierarchy in mind, it reads differently.

Tools and modern practice

- NVIDIA, *Nsight Systems* and *Nsight Compute* documentation. The profiling workflow from section 6.
- Tillet, Kung & Cox (2019), *Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations*, MAPL — https://triton-lang.org. Where most new GPU kernels are written today. Learn it after CUDA, not instead of it.
- Jia et al. (2018), *Dissecting the NVIDIA Volta GPU Architecture via Microbenchmarking*. arXiv:1804.06826. The source of the latency figures in section 3.

This is the end of the course. Its running thread, one function seen from ten angles, is now complete: tensors, mathematics, implementation, debugging, distribution, inference, retrieval, research, alignment and, finally, the GPU itself.